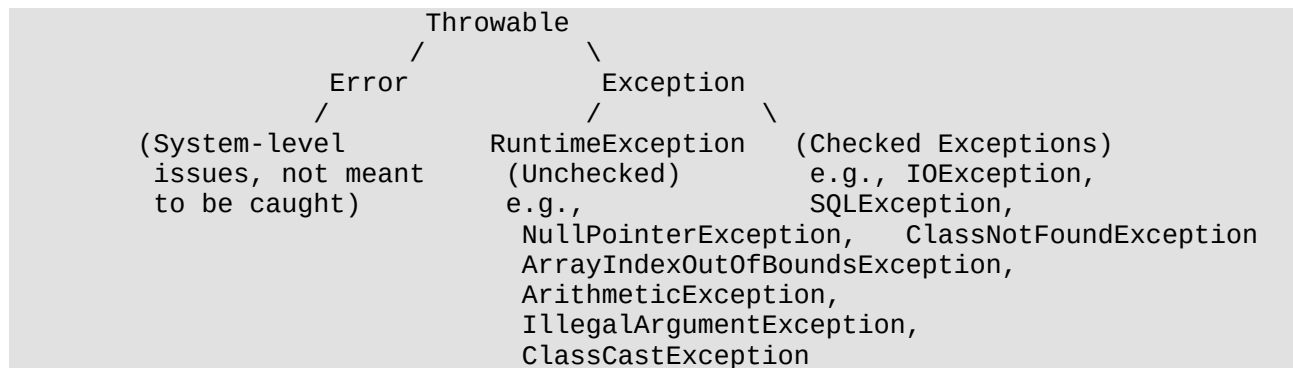


Global Exception Handling & Exception Hierarchy — Detailed Notes

1. Exception Hierarchy in Java

Root Class: Throwable

Everything that can be "thrown" in Java originates from `java.lang.Throwable`.



A) Error

- Represents **serious problems** that a normal application should **not** try to catch.
- Usually related to the JVM/environment, not application logic.
- **Examples:** `OutOfMemoryError`, `StackOverflowError`, `VirtualMachineError`, `NoClassDefFoundError`
- **Not recoverable** — the application typically cannot do anything meaningful once these occur.

B) Exception

Divided into two categories:

i) Checked Exceptions

- Checked **at compile-time** — the compiler forces you to either handle them (`try-catch`) or declare them (`throws`).
- Represent conditions a well-written application should anticipate and recover from.
- **Examples:** `IOException`, `SQLException`, `ClassNotFoundException`, `FileNotFoundException`

ii) Unchecked Exceptions (`RuntimeException`)

- **Not checked** at compile-time — the compiler does not force handling.
- Usually caused by **programming errors** (bad logic, invalid input, etc.)

- **Examples:**
 - `NullPointerException` — accessing a member on a `null` object
 - `ArrayIndexOutOfBoundsException` — invalid array index
 - `ArithmeticException` — e.g., divide by zero
 - `ClassCastException` — invalid typecasting
 - `IllegalArgumentException` — invalid argument passed to a method
 - `NumberFormatException` — invalid string-to-number conversion

Quick Comparison Table:

Aspect	Checked Exception	Unchecked Exception (Runtime)	Error
Compile-time check	Yes	No	No
Must declare/handle	Yes (throws/try-catch)	No (optional)	No
Cause	External conditions (I/O, DB, network)	Programming bugs/logic errors	JVM/system-level failure
Recoverable	Usually yes	Sometimes	Usually no
Example	<code>IOException</code> , <code>SQLException</code>	<code>NullPointerException</code> , <code>ArithmeticException</code>	<code>OutOfMemoryError</code>

2. Why Exception Handling Matters in Spring Boot (REST APIs)

In a REST API, if an exception is **not handled**, Spring Boot's default behavior returns a generic, unstructured error response (via **Whitelabel Error Page** for browsers, or a plain JSON stack-trace-like error for API clients) — which:

- Exposes internal details (bad for security)
- Is inconsistent in format
- Gives the client no clear/structured way to understand what went wrong

Global Exception Handling solves this by providing a **centralized, consistent, and clean error response structure** across the entire application — instead of writing `try-catch` in every single Controller method.

3. Key Annotations for Global Exception Handling

A) @ExceptionHandler

- Placed on a method to define **how a specific exception type should be handled**.
- Can be placed:
 - Inside a Controller (handles exceptions only for that Controller)
 - Inside a @ControllerAdvice class (handles exceptions **globally**, across all Controllers)

```
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<ErrorResponse> handleUserNotFound(UserNotFoundException
ex) {
    ErrorResponse error = new ErrorResponse(HttpStatus.NOT_FOUND.value(),
ex.getMessage());
    return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
}
```

B) @ControllerAdvice

- A class-level annotation that makes exception handling **global** — it applies to **all Controllers** in the application, not just one.
- Internally, it is a specialized @Component, so Spring detects and registers it automatically at startup.
- Can also be scoped using attributes like basePackages, annotations, or assignableTypes to limit which Controllers it applies to.

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<ErrorResponse>
handleUserNotFound(UserNotFoundException ex) {
        ErrorResponse error = new ErrorResponse(HttpStatus.NOT_FOUND.value(),
ex.getMessage());
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGenericException(Exception ex) {
        ErrorResponse error = new
ErrorResponse(HttpStatus.INTERNAL_SERVER_ERROR.value(), "Something went wrong");
        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

C) @RestControllerAdvice

- Combination of @ControllerAdvice + @ResponseBody.
- Used when building **REST APIs** — ensures the return value is automatically serialized into JSON (instead of being resolved as a view name).
- This is the **most commonly used annotation** for global exception handling in Spring Boot

REST applications.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<ErrorResponse>
    handleUserNotFound(UserNotFoundException ex) {
        ErrorResponse error = new ErrorResponse(HttpStatus.NOT_FOUND.value(),
        ex.getMessage());
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }
}
```

D) @ResponseStatus

- Can be placed directly on a custom exception class to define the default HTTP status when that exception is thrown (works without even needing `@ExceptionHandler`, though combining both gives more control).

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(String message) {
        super(message);
    }
}
```

4. Custom Exception Classes

It's a best practice to create **custom exception classes** for meaningful business errors instead of relying only on generic exceptions.

```
public class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(String message) {
        super(message);
    }
}
```

- Usually extends `RuntimeException` (unchecked) — so it doesn't force `throws` declarations everywhere, keeping code clean.
- Thrown from Service/Controller layer when a specific business condition fails:

```
public User getUserById(Long id) {
    return userRepository.findById(id)
        .orElseThrow(() -> new UserNotFoundException("User not found with id: "
+ id));
}
```

5. Standard Error Response Structure

A common practice is to define a POJO representing a consistent error response format:

```
public class ErrorResponse {
```

```

private int statusCode;
private String message;
private LocalDateTime timestamp;

// constructors, getters, setters
}

```

Example JSON output sent to the client:

```

{
  "statusCode": 404,
  "message": "User not found with id: 5",
  "timestamp": "2026-07-13T10:15:30"
}

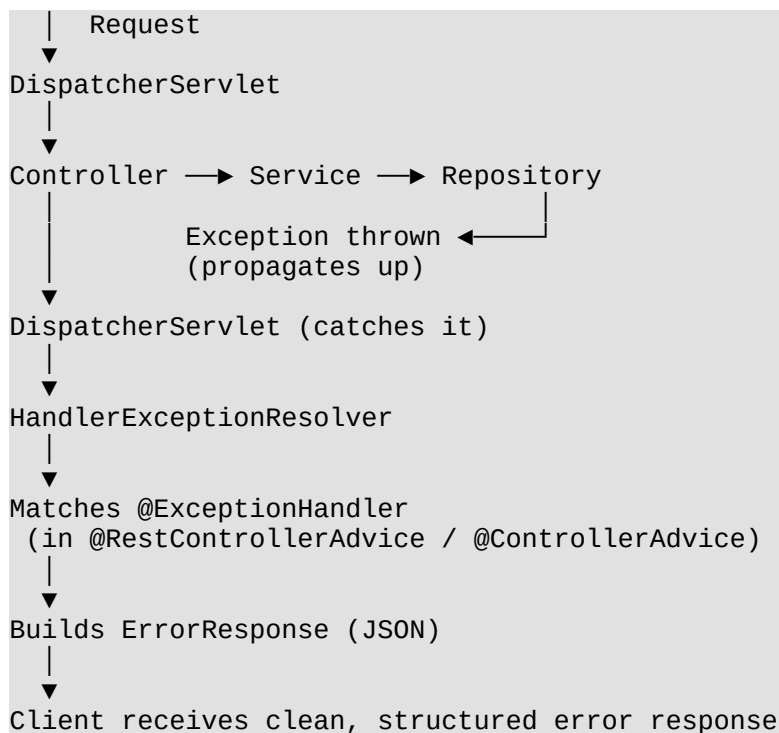
```

6. Full Flow of Global Exception Handling in Spring Boot

1. Client sends HTTP Request
 - ↓
2. Request reaches DispatcherServlet (Front Controller)
 - ↓
3. DispatcherServlet routes it to the appropriate Controller method
 - ↓
4. Controller calls Service layer → Service calls Repository layer
 - ↓
5. Somewhere in this chain, an Exception is thrown
(e.g., UserNotFoundException, NullPointerException, custom business exception)
 - ↓
6. Exception propagates UP the call stack
(Repository → Service → Controller)
since it's not caught anywhere locally
 - ↓
7. DispatcherServlet catches this unhandled exception
(it wraps handler execution in try-catch internally)
 - ↓
8. DispatcherServlet delegates to HandlerExceptionResolver
 - ↓
9. Spring checks for a matching @ExceptionHandler method:
 - a) First checks LOCAL @ExceptionHandler in the same Controller
 - b) If not found, checks GLOBAL @ControllerAdvice / @RestControllerAdvice classes
 - c) Matches based on the MOST SPECIFIC exception type first, then falls back to more generic types (e.g., Exception.class)
 - ↓
10. Matching @ExceptionHandler method executes
 - ↓
11. Builds a structured ErrorResponse object (status code, message, timestamp, etc.)
 - ↓
12. Returned via ResponseEntity<> (converted to JSON by HttpMessageConverter)
 - ↓
13. DispatcherServlet sends this structured response back
 - ↓
14. Response reaches Client with proper HTTP status code and clean JSON body

Visual Flow Diagram

Client



7. Matching Priority for @ExceptionHandler

When multiple `@ExceptionHandler` methods could apply, Spring resolves them in this order:

1. **Local handler in the same Controller** takes priority over global (`@ControllerAdvice`) handlers.
2. **Most specific exception type** is matched first.
 - E.g., if both `handleUserNotFoundException(UserNotFoundException ex)` and `handleGenericException(Exception ex)` exist, and a `UserNotFoundException` is thrown, the **specific one** (`UserNotFoundException`) is called — not the generic `Exception` handler.
3. If no specific match is found, it falls back to the broader `Exception.class` handler (a good practice to always have one as a catch-all).

8. Handling Validation Exceptions (Common Real-World Case)

When using `@Valid` with request bodies (Bean Validation), failed validations throw `MethodArgumentNotValidException`. This is commonly handled globally too:

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, String>>
handleValidationErrors(MethodArgumentNotValidException ex) {
    Map<String, String> errors = new HashMap<>();
    ex.getBindingResult().getFieldErrors().forEach(error ->
        errors.put(error.getField(), error.getDefaultMessage())
    );
    return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
}
```

```
}
```

9. Common Built-in Spring/Java Exceptions Handled Globally

Exception	Typical Cause	Suggested HTTP Status
MethodArgumentNotValidException	@Valid validation failure on request body	400 Bad Request
HttpMessageNotReadableException	Malformed JSON in request	400 Bad Request
MissingServletRequestParameterException	Required request param missing	400 Bad Request
HttpRequestMethodNotSupportedException	Wrong HTTP method used (e.g., GET instead of POST)	405 Method Not Allowed
NoHandlerFoundException	No matching URL/Controller found	404 Not Found
DataIntegrityViolationException	DB constraint violation	409 Conflict
AccessDeniedException	Authorization failure (Spring Security)	403 Forbidden
Custom (UserNotFoundException, etc.)	Business logic-specific errors	Varies (404, 409, etc.)
Exception.class (catch-all)	Any unhandled/unexpected error	500 Internal Server Error

10. Benefits of Global Exception Handling

- **Centralization** — one place (@RestControllerAdvice) manages error handling for the entire application instead of repeating try-catch in every Controller.
- **Consistency** — every error response follows the same structured JSON format.
- **Security** — prevents leaking internal stack traces/implementation details to the client.
- **Cleaner Code** — Controllers and Services stay focused on business logic; error formatting is handled separately.
- **Maintainability** — adding a new exception type just means adding one more @ExceptionHandler method, without touching existing Controllers.

11. Quick Summary (for revision)

- **Exception Hierarchy:** Throwable → Error / Exception → (Checked / RuntimeException=Unchecked)
- **Checked exceptions** must be declared/handled at compile-time; **unchecked (Runtime) exceptions** are not compiler-enforced and usually indicate programming errors.
- **@ExceptionHandler** — defines handling logic for a specific exception type.
- **@ControllerAdvice** / **@RestControllerAdvice** — makes exception handling **global**, applying across all Controllers (@RestControllerAdvice = @ControllerAdvice + @ResponseBody, auto-returns JSON).
- **Flow:** Exception thrown anywhere in Repository/Service/Controller → propagates up → caught by DispatcherServlet → delegated to HandlerExceptionResolver → matched to the most specific @ExceptionHandler → structured ErrorResponse (JSON) returned to client.
- Always keep a **generic Exception.class handler** as a fallback for unexpected errors (mapped to 500 Internal Server Error).
- Custom exceptions (extending RuntimeException) + ErrorResponse POJO + @RestControllerAdvice = the standard, clean pattern for REST API error handling in Spring Boot.